AI DB Studio in Rust

Proposed Architecture for a Production-Grade Database Workbench with a Safe AI Assistant

A complete design for rebuilding AI DB Studio on Tauri 2 and Rust: workspace layout, crate responsibilities, the database driver layer, AST-based SQL safety, the AI agent, SSH tunnelling, storage and secrets, IPC, security, packaging, testing and a delivery roadmap.

Shell	Tauri 2.11 (system webview)
Backend	Rust (stable), Tokio async runtime
Frontend	React + TypeScript (reused from the Electron version)
Databases	PostgreSQL, MySQL/MariaDB, SQLite (extensible)
AI	Ollama, OpenAI and compatible endpoints, via `rig-core`
Platforms	macOS, Windows, Linux
Date	7 November 2026

Contents

1 Overview **4**

1.1 Goal . . . 4

1.2 Why Rust and Tauri . . . 4

1.3 Architecture at a glance . . . 4

2 System Architecture **6**

2.1 Layers . . . 6

2.2 Cargo workspace layout . . . 6

2.2.1 Crate dependency graph . . . 7

2.3 Concurrency model . . . 8

3 The Database Layer **9**

3.1 The driver abstraction (`studio-db`) . . . 9

3.1.1 Values and batches . . . 9

3.2 Engine drivers . . . 10

3.2.1 Multi-statement scripts . . . 10

3.3 Session manager (`studio-session`) . . . 11

3.4 SSH tunnels (`studio-tunnel`) . . . 11

4 SQL Safety **13**

4.1 Principles . . . 13

4.2 Classifier (`studio-safety`) . . . 13

4.3 Approval tokens . . . 14

5 The AI Assistant **16**

5.1 Structure (`studio-ai`) . . . 16

5.2 Tools . . . 16

5.3 The review gate . . . 16

5.4 Limits and privacy . . . 17

5.5 Memory . . . 17

5.6 Fallback modes . . . 18

6 Storage, Secrets and Audit **19**

6.1 App database (`studio-store`) . . . 19

6.2 Secrets . . . 19

6.3 Optional encryption at rest . . . 20
6.4 Audit log . . . 20

7 IPC Between Frontend and Rust 21
7.1 Three mechanisms . . . 21
7.2 Typed bindings . . . 21
7.3 Errors . . . 22
7.4 Command catalogue . . . 22

8 Key Flows 23
8.1 Connecting through an SSH tunnel with TLS . . . 23
8.2 Running a query with streaming results . . . 24
8.3 An AI turn with one approved write . . . 25

9 Security Architecture 26
9.1 Threat model . . . 26
9.2 Tauri capabilities . . . 26
9.3 Content Security Policy . . . 27
9.4 TLS . . . 27

10 Frontend 28
10.1 Reusing the React app . . . 28
10.2 State . . . 28
10.3 New UI for production features . . . 28

11 Production Features Map 29

12 Build, Release and Operations 30
12.1 Development . . . 30
12.2 Bundling . . . 30
12.3 Auto-update . . . 30
12.4 Continuous integration . . . 30
12.5 Observability . . . 31

13 Testing Strategy 32

14 Migration Plan and Roadmap 33
14.1 From the Electron version . . . 33
14.2 Milestones . . . 33
14.3 Risks . . . 33

A Dependency Reference **34**

B Example Workspace Manifest **35**

C Glossary **36**

CHAPTER 1

Overview

1.1 Goal

This document proposes how to build AI DB Studio — a desktop client for PostgreSQL, MySQL and other databases, with an AI assistant that can read but never change data without approval — in **Rust**, using **Tauri 2** as the desktop shell. It is written for the person who will build it, and covers every layer from the folder structure to release signing.

The design keeps everything that worked well in the Electron version (a trusted backend, a narrow typed bridge, review before writes, privacy by default) and fixes its known gaps: certificate verification, lexical rather than parsed SQL checks, fetching whole result sets into memory, no SSH tunnelling, no Linux build, unsigned installers and unencrypted local data.

1.2 Why Rust and Tauri

Concern	Electron + Node (current)	Tauri + Rust (proposed)
App size	Bundles Chromium and Node (80–180 MB installers)	Uses the OS webview; installers are typically a few MB to a few tens of MB
Memory	One Chromium plus Node process	One Rust process plus the OS webview
Large results	Rows parsed into JS objects; capped after fetch	Rows streamed from the socket in batches; constant memory
Concurrency	Single-threaded event loop	Multi-threaded Tokio runtime; queries never block the UI
Safety of core logic	Dynamic types; runtime errors	Compile-time types, exhaustive `match`, no null
Permissions	Preload exposes everything it defines	Tauri capabilities grant each command explicitly
Learning curve	Low	Higher (ownership, async, lifetimes)

Table 1.1: Trade-offs. Sizes are typical ranges, not measurements of this app.

1.3 Architecture at a glance

Core decisions

1. **Tauri 2** shell; **React + TypeScript** frontend reused from the current app, with `window.electronAPI` replaced by generated, typed bindings.
2. **A Cargo workspace of small crates**, each with one job. The Tauri app crate is a thin layer of commands on top of them, so all logic can be unit-tested without a window.
3. **A `Driver` trait** with one implementation per engine: `tokio-postgres`, `mysql_async`, `rusqlite`. Results *stream* to the UI over Tauri `Channels`.
4. **AST-based SQL safety** with `sqlparser`, failing closed, backed by database-enforced read-only transactions and read-only connections.

5. **Approval tokens** bound to a hash of the exact SQL and session, single-use, short-lived.
6. **AI agent** built on `rig-core` for provider access, with our own loop so review gating, limits and privacy rules stay in our code.
7. **Local SQLite database** for app data (with migrations), **OS keychain** for secrets via `keyring`, optional SQLCipher encryption.
8. **SSH tunnels** via `russh`, with host-key verification.
9. **rustls with certificate verification on by default**.
10. **Signed builds and auto-update** through `tauri-plugin-updater`; macOS, Windows and Linux bundles from CI.

CHAPTER 2

System Architecture

2.1 Layers

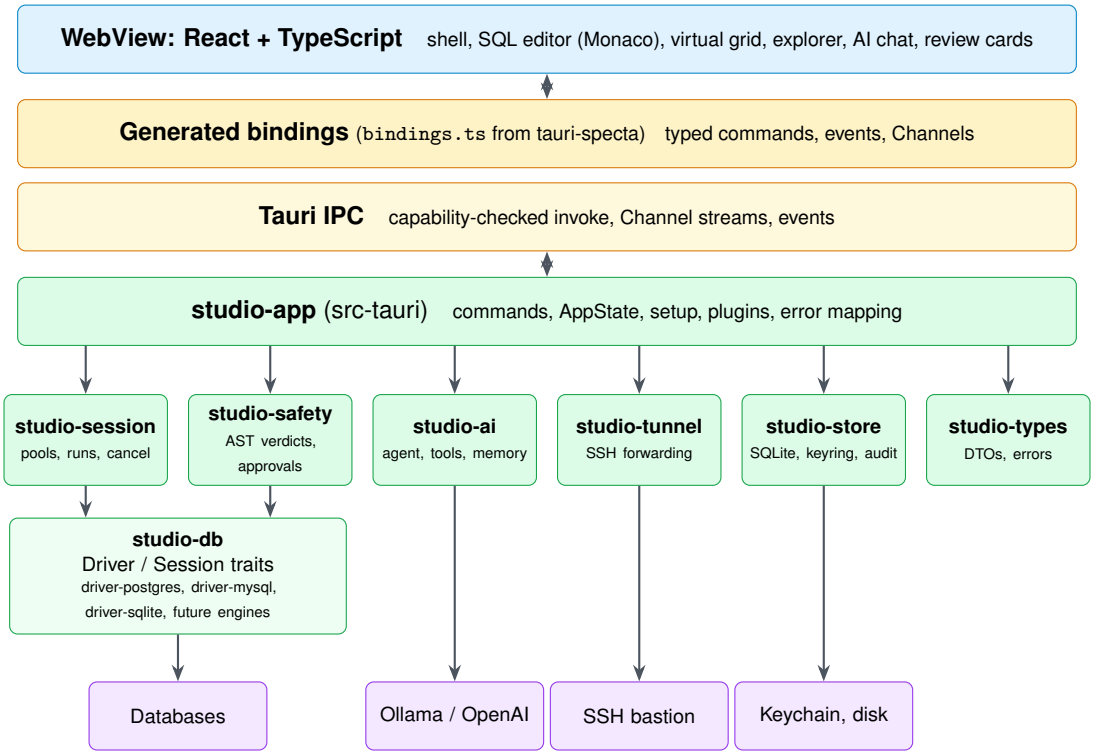


Figure 2.1: Layered architecture. Arrows point from caller to callee.

The rule that makes this safe is the same as before, enforced more strictly:

Trust boundary

The **webview is untrusted**. It cannot open sockets, read files or see secrets. It may only call the commands that the app's capability file grants it, and every command re-validates its input. Everything sensitive — drivers, passwords, approval decisions, the AI agent — lives in the Rust process.

2.2 Cargo workspace layout

```
ai-db-studio/
|-- Cargo.toml                  [workspace] members, shared [workspace.dependencies]
|-- rust-toolchain.toml         pin stable channel + components (clippy, rustfmt)
|-- deny.toml                   cargo-deny: licences, advisories, duplicate crates
|-- crates/
|   |-- studio-types/           serde + specta DTOs shared with the frontend, AppError
|   |-- studio-db/              Driver/Session traits, Value, RowBatch, DbError
|   |-- driver-postgres/        tokio-postgres + deadpool-postgres + rustls
```

```
|   |-- driver-mysql/              mysql_async + rustls
|   |-- driver-sqlite/             rusqlite (bundled) on a blocking thread
|   |-- studio-safety/             sqlparser classifier, risk analysis, approval tokens
|   |-- studio-session/            SessionManager: connect, run, stream, cancel, tx mode
|   |-- studio-tunnel/             russh local port forwarding, host-key checks
|   |-- studio-ai/                 agent loop, tools, providers (rig-core), memory, fallback
|   `-- studio-store/              app SQLite (sqlx + migrations), keyring, audit log
|-- src-tauri/                     studio-app: the Tauri binary
|   |-- Cargo.toml
|   |-- tauri.conf.json            bundle, CSP, updater config
|   |-- capabilities/main.json     which commands/plugins the main window may use
|   |-- icons/
|   `-- src/
|       |-- main.rs                calls studio_app::run()
|       |-- lib.rs                 Builder, plugins, state, specta export
|       |-- state.rs               AppState
|       |-- error.rs               AppError -> serializable error for the UI
|       `-- commands/              profiles.rs, sessions.rs, query.rs, schema.rs,
|                                  ai.rs, history.rs, settings.rs, export.rs, audit.rs
|-- src/                           React app (ported from the Electron renderer)
|   |-- bindings.ts                GENERATED by tauri-specta (do not edit)
|   |-- api/                       thin wrappers + Channel helpers
|   |-- context/  components/  lib/
|-- tests/e2e/                     WebDriver tests via tauri-driver
`-- .github/workflows/             ci.yml (lint, test), release.yml (tauri-action)
```

2.2.1 Crate dependency graph

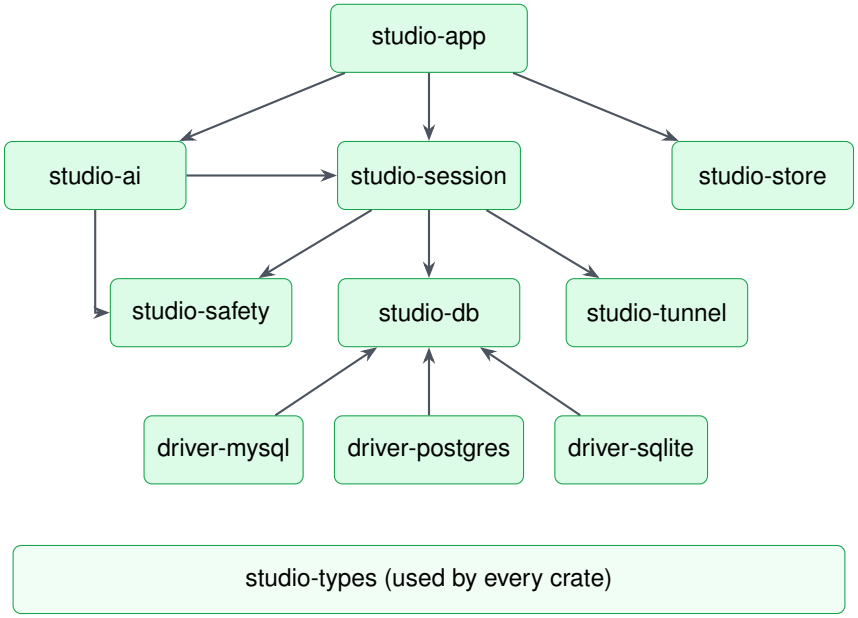


Figure 2.2: Dependencies. Drivers depend only on `studio-db`; the app wires the concrete drivers in.

Two properties of this graph matter:

- **No driver knows about Tauri, AI or storage.** You can test `driver-postgres` against a container with `cargo test -p driver-postgres`.
- `studio-ai` **cannot reach a database except through** `studio-session`, which applies the safety rules. There is no code path from the model to a raw connection.

2.3 Concurrency model

- Tauri runs on a multi-threaded **Tokio** runtime. Every command is an `async fn`.
- Each running query is a spawned task holding a `CancellationToken` (from `tokio-util`). Cancelling a tab cancels its token and sends the engine's cancel request.
- Blocking work — `rusqlite`, `keyring`, file export — runs inside `tokio::task::spawn_blocking` so it never stalls the async threads.
- Shared state lives in `AppState` behind `Arc`. Maps use `dashmap::DashMap` or `tokio::sync::RwLock`; no lock is held across an `.await` on network I/O.

```rust
pub struct AppState {
    pub store:     studio_store::Store,                 // app SQLite + keyring
    pub sessions:  studio_session::SessionManager,      // open connections and runs
    pub approvals: studio_safety::ApprovalBook,         // pending write approvals
    pub ai:        studio_ai::AgentHub,                 // active AI turns + pending reviews
    pub settings:  tokio::sync::RwLock<Settings>,
}
```

CHAPTER 3

The Database Layer

3.1 The driver abstraction (`studio-db`)

Every engine implements two traits. Using `async_trait` keeps them object-safe, so the session manager can hold a `Box<dyn Session>` regardless of engine.

```rust
#[async_trait::async_trait]
pub trait Driver: Send + Sync + 'static {
    fn engine(&self) -> Engine;
    fn dialect(&self) -> Dialect;                          // used by studio-safety
    async fn connect(&self, cfg: &ConnectConfig) -> Result<Box<dyn Session>, DbError>;
}

#[async_trait::async_trait]
pub trait Session: Send + Sync {
    async fn server_info(&self) -> Result<ServerInfo, DbError>;
    async fn list_schemas(&self) -> Result<Vec<SchemaRef>, DbError>;
    async fn list_objects(&self, schema: &SchemaRef) -> Result<Vec<DbObject>, DbError>;
    async fn columns(&self, schema: &SchemaRef) -> Result<Vec<ColumnInfo>, DbError>;
    async fn table_structure(&self, t: &TableRef) -> Result<TableStructure, DbError>;

    /// Runs one statement and streams rows into `sink` in batches.
    async fn execute(&self, stmt: &str, opts: ExecOptions, sink: RowSink)
        -> Result<ExecSummary, DbError>;

    /// Starts/ends a manual transaction for "manual commit" mode.
    async fn begin(&self, read_only: bool) -> Result<(), DbError>;
    async fn commit(&self) -> Result<(), DbError>;
    async fn rollback(&self) -> Result<(), DbError>;

    /// A cheap, cloneable handle that can cancel whatever is running.
    fn canceller(&self) -> Arc<dyn Canceller>;
    async fn close(&self);
}

pub struct ExecOptions {
    pub read_only_tx: bool,          // wrap in a READ ONLY transaction, then roll back
    pub timeout: Option<Duration>,   // statement_timeout / MAX_EXECUTION_TIME
    pub max_rows: Option<u64>,       // stop reading after N rows (server keeps cursor small)
    pub batch_size: usize,           // rows per RowBatch sent to the UI, e.g. 500
}
```

3.1.1 Values and batches

Cell values use one engine-neutral enum. Types JavaScript cannot represent exactly (64-bit integers, decimals, timestamps) travel as strings with a type tag, so nothing is silently rounded and no timezone conversion happens — the same lesson the Electron version learned with dates.

```rust
#[derive(Serialize, specta::Type)]
#[serde(tag = "t", content = "v")]
pub enum Value {
    Null,
```

```
    Bool(bool),
    Int(i32),               // fits a JS number
    BigInt(String),         // i64 / u64 / numeric as text
    Float(f64),
    Decimal(String),
    Text(String),
    Json(String),
    Bytes { len: usize, preview_hex: String },   // never ship huge blobs to the grid
    Temporal(String),       // database's own text form, no tz conversion
    Unsupported(String),    // type name, for display
}

pub struct RowBatch { pub seq: u32, pub rows: Vec<Vec<Value>> }
pub type RowSink = tokio::sync::mpsc::Sender<RowBatch>;
```

3.2 Engine drivers

	PostgreSQL	MySQL / MariaDB	SQLite
Crate	`tokio-postgres` + `deadpool-postgres`	`mysql_async`	`rusqlite` (bundled)
TLS	`tokio-postgres-rustls`, verified by default	built-in rustls feature, verified by default	n/a (local file)
Streaming	`query_raw` returns a `RowStream`	`query_iter` result stream	step through `Rows` on a blocking thread
Cancel	`CancelToken::cancel_query`	`KILL QUERY <conn id>` on a second connection	`interrupt_handle().interrupt()`
Read-only	`BEGIN READ ONLY` + `SET LOCAL statement_timeout`	`START TRANSACTION READ ONLY` + `MAX_EXECUTION_TIME`	open with `SQLITE_OPEN_READ_ONLY`
Metadata	`pg_catalog` + `information_schema`	`information_schema`, `SHOW CREATE`	`sqlite_schema`, `pragma_table_info`

Table 3.1: Driver choices per engine.

Why not sqlx for user databases?

`sqlx` is excellent for an application's *own* schema, and this design uses it for the local app database. For arbitrary user SQL, however, a database client needs things that are more direct in the native drivers: protocol-level cancellation tokens, row streaming with column type metadata for any query shape, fine control over transactions on a pinned connection, and easy access to the server's process or connection id. `tokio-postgres` and `mysql_async` expose all of these. Hiding them behind `studio-db` means the choice can change later without touching the app.

3.2.1 Multi-statement scripts

Rather than sending a whole script and receiving only the last result (as the Electron version does), the session manager splits the script with `sqlparser`, classifies each statement, and runs them in order on one pinned connection. The UI receives one result tab per statement that returns rows, and a summary line per command. If a statement fails, execution stops and earlier results stay visible. In manual-commit mode all statements share the open transaction.

3.3 Session manager (`studio-session`)

The session manager owns every open connection and every running query.

```
pub struct SessionManager {
    drivers:  HashMap<Engine, Arc<dyn Driver>>,
    sessions: DashMap<SessionId, Arc<OpenSession>>,
    runs:     DashMap<RunId, RunHandle>,
}

pub struct OpenSession {
    pub profile:  ConnectionProfile,          // no secrets
    pub session:  Box<dyn Session>,
    pub tunnel:   Option<studio_tunnel::Tunnel>,  // dropped => tunnel closes
    pub policy:   SessionPolicy,               // read_only, environment (prod/staging/dev)
    pub tx_state: tokio::sync::Mutex<TxState>, // Auto | Manual { dirty: bool }
}

pub struct RunHandle {
    pub session: SessionId,
    pub cancel:  tokio_util::sync::CancellationToken,
    pub canceller: Arc<dyn Canceller>,
    pub started: Instant,
}
```

Connection policy. Each profile carries an `environment` (`Development`, `Staging`, `Production`) and a `read_only` flag. For read-only profiles the driver connects with a read-only default where the engine supports it (for PostgreSQL, `default_transaction_read_only = on` in the startup options), and the session manager refuses any statement the classifier does not mark as a read — even with an approval. The frontend colours the title bar by environment.

Execution entry points. As in the current app there are separate doors with different rules:

Method	Gate	Row cap	Timeout
`run_user`	classify every statement; writes need an approval token; read-only profiles refuse writes	configurable (streamed)	optional
`run_ai_read`	strict AI verdict *and* read-only transaction; no approval path	5,000	30 s
`run_ai_approved`	approval token minted by a human review, bound to the SQL	5,000	60 s
`explain`	single statement; `EXPLAIN (FORMAT JSON)` for the plan viewer	n/a	30 s

Table 3.2: Execution paths in `studio-session`.

3.4 SSH tunnels (`studio-tunnel`)

1. Connect to the bastion with `russh::client`, verifying the host key against the app's own `known_hosts` store (trust on first use with an explicit confirmation dialog showing the fingerprint; a changed key is a hard error).
2. Authenticate with a private key (optionally passphrase-protected, or via the SSH agent), keyboard-interactive or password.

3. Bind a `TcpListener` on `127.0.0.1:0` (a random free port).
4. For every accepted local socket, open a `direct-tcpip` channel to the database host and port, and copy bytes both ways with `tokio::io::copy_bidirectional`.
5. The driver connects to `127.0.0.1:<port>`. Dropping the `Tunnel` value closes the listener and the SSH session.

```
pub struct Tunnel { pub local_port: u16, shutdown: CancellationToken, task: JoinHandle<()> }

impl Drop for Tunnel {
    fn drop(&mut self) { self.shutdown.cancel(); }   // RAII: no leaked forwards
}
```

Keep-alives are sent every 30 seconds; if the SSH session drops, the session is marked `Disconnected` and the UI offers to reconnect.

CHAPTER 4

SQL Safety

4.1 Principles

1. **Parse, don't guess.** Use `sqlparser` with the right dialect (`PostgreSqlDialect`, `MySqlDialect`, `SQLiteDialect`).
2. **Fail closed.** If the SQL does not parse, it is treated as a possible write: users get a confirmation dialog, the AI gets a review card.
3. **Defence in depth.** The parser decides who must approve; the database enforces read-only transactions for automatic AI reads; read-only profiles are enforced in the session manager.
4. **Bind approvals to the exact text.** An approval covers one hash of one SQL string on one session, once.

4.2 Classifier (`studio-safety`)

```
pub enum StatementKind { Select, Show, Explain, Insert, Update, Delete, Merge, Ddl, Dcl, Tcl,
    Other }

pub struct Verdict {
    pub kind: StatementKind,
    pub read_only: bool,          // may run without a human (user path)
    pub ai_auto: bool,            // may run with no human for the AI (strict)
    pub risk: Risk,               // Normal | High { reasons: Vec<String> }
    pub reason: Option<String>,   // why ai_auto is false
}

pub fn classify_script(sql: &str, d: Dialect) -> Result<Vec<(Span, Verdict)>, ParseFailure>;
```

The classifier walks the AST (enable the crate's `visitor` feature) and applies these rules:

Construct	read_only	ai_auto	Notes
SELECT, SHOW, DESCRIBE, EXPLAIN (no ANALYZE)	yes	yes	default safe reads
SELECT ... INTO	no	no	creates a table or writes a file
SELECT ... FOR UPDATE / SHARE	yes	no	takes locks
WITH containing INSERT/UPDATE/DELETE/MERGE	no	no	data-modifying CTE
EXPLAIN ANALYZE <stmt>	as <stmt>	as <stmt>	really executes
Calls to side-effect functions (deny-list per dialect)	yes	no	pg_sleep, nextval, dblink, lo_*, pg_read_file, sleep, benchmark, get_lock, load_file ...
INSERT, UPDATE, DELETE, MERGE	no	no	High risk if no WHERE
CREATE, ALTER, DROP, TRUNCATE, RENAME	no	no	DROP/TRUNCATE High risk
GRANT, REVOKE, SET ROLE	no	no	privilege changes
BEGIN/COMMIT/ROLLBACK	handled by tx mode	no	AI may not control transactions
MySQL executable comments /*! ... */	no	no	detected by a pre-lexer pass
Parse failure	no	no	fail closed
More than one statement	per statement	no	the AI may submit one statement at a time

Table 4.1: Classification rules.

```
// Sketch of the strict AI check. Exact sqlparser type names vary by version.
pub fn ai_auto_allowed(sql: &str, d: Dialect) -> Result<(), String> {
    if lexer::has_executable_comment(sql) { return Err("Executable comments are not
    allowed".into()); }
    let stmts = Parser::parse_sql(d.as_sqlparser(), sql).map_err(|_| "Could not
    parse".to_string())?;
    let [stmt] = stmts.as_slice() else { return Err("Exactly one statement is allowed".into())
    };
    let v = verdict_for(stmt, d);
    if !v.ai_auto { return Err(v.reason.unwrap_or_else(|| "Not a read".into())); }
    Ok(())
}
```

Testing the classifier. Keep a table-driven test file with several hundred cases per dialect (port the existing `readonly.test.mjs` cases first), plus a `cargo-fuzz` target that asserts the classifier never panics and that any statement it marks `ai_auto` is rejected by a real database inside a read-only transaction when it tries to write.

4.3 Approval tokens

```
pub struct ApprovalBook { pending: DashMap<TokenHash, Pending> }

struct Pending { session: SessionId, sql_hash: [u8; 32], expires: Instant, origin: Origin }

impl ApprovalBook {
    /// Called only from the command that handles a human's Confirm/Approve click.
    pub fn issue(&self, session: SessionId, sql: &str, origin: Origin) -> ApprovalToken {
        let token = ApprovalToken::random();                    // 32 bytes from OsRng
        self.pending.insert(token.hash(), Pending {
```

```
            session, sql_hash: sha256(sql), expires: Instant::now() + Duration::from_secs(60),
    origin,
        });
        token
    }
    /// Removes the token first, then checks it: single use even on mismatch.
    pub fn consume(&self, t: &ApprovalToken, session: SessionId, sql: &str) -> Result<Origin,
    SafetyError> {
        let (_, p) = self.pending.remove(&t.hash()).ok_or(SafetyError::NoApproval)?;
        if p.expires < Instant::now() { return Err(SafetyError::Expired); }
        if p.session != session || p.sql_hash != sha256(sql) { return
    Err(SafetyError::Mismatch); }
        Ok(p.origin)
    }
}
```

Only the token's hash is stored, the raw token never appears in logs, and every consumed approval writes an audit record (Chapter 6).

CHAPTER 5

The AI Assistant

5.1 Structure (`studio-ai`)

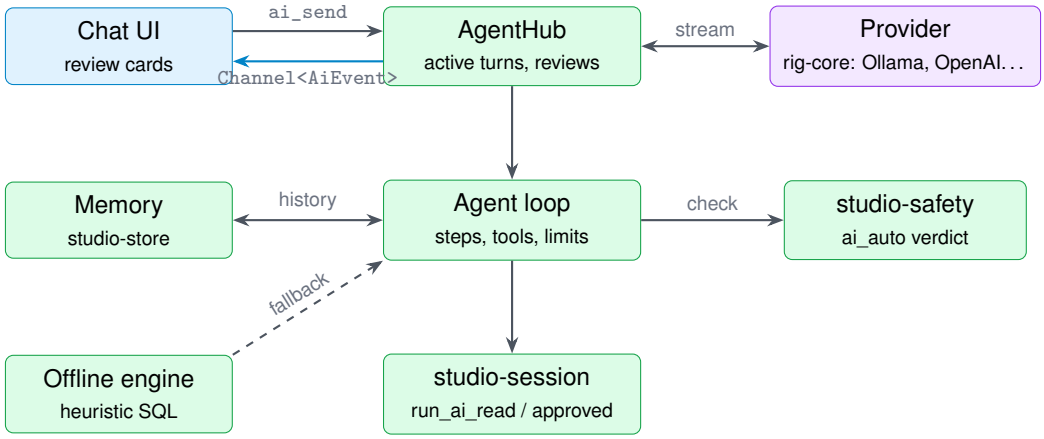


Figure 5.1: AI subsystem. The model reaches the database only through the loop's `run_sql` tool.

Use rig-core for providers, keep the loop ourselves

`rig-core` gives one Rust interface over many providers (OpenAI, Ollama, Anthropic, Gemini, OpenRouter and others), with streaming and tool definitions. Its built-in agent can run tools automatically, but this app needs to *pause* a tool call for human review, enforce per-turn limits, and control exactly what text goes back to the model. So the design uses rig's completion models and tool schemas, and drives the multi-step loop in `studio-ai`. If rig's API changes, only the `Provider` adapter moves.

5.2 Tools

Tool	Behaviour
`run_sql {sql}`	One statement. If `ai_auto`, runs via `run_ai_read`; otherwise emits a review card and waits. Returns shape only, or a capped sample if result sharing is on.
`describe_table {schema?, table}`	Columns, types, keys, indexes. Never rows.
`explain_query {sql}`	Plain `EXPLAIN` (never ANALYZE) for optimisation advice.
`list_tables {schema?}`	Names only; used when the schema was too large to fit in the prompt.

Table 5.1: Tools exposed to the model.

5.3 The review gate

Review uses a Tokio `oneshot` channel whose sender is stored in the main process. Only the `ai_review` command — triggered by a click on the review card — can complete it.

```
async fn handle_run_sql(&self, turn: &Turn, sql: String) -> String {
    if let Err(e) = turn.limits.admit(&sql) { return e.to_string(); }   // dup / max 6 per turn
    let id = QueryId::new();
    let auto = ai_auto_allowed(&sql, turn.dialect);
    let needs_review = auto.is_err() || !turn.auto_run;
    turn.events.send(AiEvent::QueryStart { id, sql: sql.clone(),
        read_only: auto.is_ok(), reason: auto.as_ref().err().cloned(), needs_review });

    if needs_review {
        let (tx, rx) = tokio::sync::oneshot::channel::<bool>();
        self.reviews.insert((turn.id, id), tx);
        let approved = tokio::select! {
            r = rx => r.unwrap_or(false),
            _ = turn.cancel.cancelled() => false,
        };
        if !approved {
            turn.events.send(AiEvent::QueryDeclined { id });
            return "The user declined this statement. It did not run. Do not submit it
    again.".into();
        }
    }
    let result = match auto {
        Ok(()) => self.sessions.run_ai_read(turn.session, &sql, id).await,
        Err(_) => self.sessions.run_ai_approved(turn.session, &sql, id).await, // token minted
    at approval
    };
    describe_for_model(result, turn.share_rows)   // rows stay in the UI unless sharing is on
}
```

5.4 Limits and privacy

Rule	Value
Model steps per turn	8
Distinct statements per turn	6; identical statement never re-run in the same turn
Schema in prompt	up to ~30,000 characters, then `list_tables` / `describe_table`
Rows to the model	none by default; opt-in sample of 50 rows / 8,000 characters, labelled untrusted
Rows in memory	never
Masked columns	columns tagged sensitive (e.g. `email`, `phone`, `*password*`) are replaced with *** before any sharing
Per-profile AI policy	allow cloud / local-only / disabled; production profiles default to local-only or no row sharing
Credentials	never in prompts; API keys only in the keychain

Table 5.2: AI guardrails.

5.5 Memory

Conversation memory moves from a JSON file to the app database: tables `conversations` and `turns`, each turn storing the user's words, the assistant's words and a list of statements with their outcomes — never rows. Compaction works as today: when a conversation exceeds a size budget, older turns are summarised (in the background, after the reply) and replaced by the summary only if the summary succeeds. Regenerate deletes from a turn onward.

5.6 Fallback modes

- **No tool calling**: if the provider rejects tools, switch to fenced "‘`sql` blocks; the same `handle_run_sql` path runs them. Remember the model so later turns skip the failed attempt.
- **Model unreachable**: the offline heuristic engine (ported from `AiService.ts`) answers simple reads, fed through the same loop with a stand-in model.

CHAPTER 6

Storage, Secrets and Audit

6.1 App database (`studio-store`)

All local data lives in one SQLite file, `studio.db`, in the platform data directory (resolved with Tauri's path API). `sqlx` with the SQLite driver gives compile-time checked queries and embedded migrations (`sqlx::migrate!`). WAL mode is enabled.

```
CREATE TABLE profiles (
  id TEXT PRIMARY KEY, name TEXT NOT NULL, engine TEXT NOT NULL,
  host TEXT, port INTEGER, database TEXT, username TEXT,
  tls_mode TEXT NOT NULL DEFAULT 'verify-full', ca_cert_path TEXT,
  ssh_json TEXT,                  -- bastion host/port/user/auth kind, no secrets
  environment TEXT NOT NULL DEFAULT 'development', read_only INTEGER NOT NULL DEFAULT 0,
  ai_policy TEXT NOT NULL DEFAULT 'default', color TEXT,
  created_at INTEGER NOT NULL, updated_at INTEGER NOT NULL
);
CREATE TABLE drafts   (id TEXT PRIMARY KEY, profile_id TEXT, title TEXT, sql TEXT, saved_sql
    TEXT, updated_at INTEGER);
CREATE TABLE history  (id TEXT PRIMARY KEY, profile_id TEXT, sql TEXT, started_at INTEGER,
                       duration_ms INTEGER, row_count INTEGER, status TEXT);
CREATE TABLE settings (key TEXT PRIMARY KEY, value_json TEXT NOT NULL);
CREATE TABLE conversations (id TEXT PRIMARY KEY, profile_id TEXT, title TEXT, summary TEXT,
    updated_at INTEGER);
CREATE TABLE turns    (id TEXT PRIMARY KEY, conversation_id TEXT REFERENCES conversations ON
    DELETE CASCADE,
                       seq INTEGER, user_text TEXT, assistant_text TEXT, queries_json TEXT);
CREATE TABLE audit_log (id INTEGER PRIMARY KEY AUTOINCREMENT, at INTEGER NOT NULL, profile_id
    TEXT,
                        environment TEXT, origin TEXT, kind TEXT, sql_hash TEXT, sql_text TEXT,
                        rows_affected INTEGER, outcome TEXT, prev_hash TEXT, entry_hash TEXT);
CREATE TABLE known_hosts (host TEXT, port INTEGER, key_type TEXT, fingerprint TEXT,
                          PRIMARY KEY (host, port, key_type));
```

6.2 Secrets

- Database passwords, SSH key passphrases and AI API keys go into the OS credential store with the `keyring` crate (features `apple-native`, `windows-native`, `sync-secret-service`). The service name is the app id; the entry name is `profile:<id>:password`, `profile:<id>:ssh`, `ai:<provider>`.
- `keyring` calls are synchronous and, on Linux, go over D-Bus; run them through one dedicated blocking worker to avoid concurrent access problems.
- Secrets are read only when connecting, wrapped in `secrecy::SecretString` (zeroed on drop), and **never sent to the webview**. The profile editor shows "password saved" and offers "replace" rather than displaying the stored value — a change from the Electron version.
- If no credential store is available, secrets are kept in memory for the session only.

6.3 Optional encryption at rest

To protect saved SQL, history and conversations, build `studio.db` with SQLCipher (`libsqlite3-sys` with the `bundled-sqlcipher` feature) and keep a random 256-bit database key in the keychain. This is a settings toggle, off by default for portability.

6.4 Audit log

Every statement that is not a pure read, from any origin (editor, grid edit, AI-approved), writes a row: time, profile, environment, origin, kind, SQL hash and text (configurable redaction), rows affected and outcome. Each entry stores the hash of the previous entry (`prev_hash`), making the log tamper-evident. Users can filter and export it as CSV or JSON.

CHAPTER 7

IPC Between Frontend and Rust

7.1 Three mechanisms

Mechanism	Use	Example
Commands (`invoke`)	Request/response	`list_profiles`, `connect`, `table_structure`
Channels (`Channel<T>`)	Ordered, high-throughput stream for one call	row batches of a query; AI events of one turn
Events (`emit`)	App-wide notifications	connection dropped, update available, menu command

Table 7.1: Tauri IPC mechanisms and where each is used.

7.2 Typed bindings

Every command and DTO derives `specta::Type`; `tauri-specta` writes `src/bindings.ts` in debug builds. The frontend imports `commands` and `events` from it, so a renamed Rust field breaks the TypeScript build instead of failing at run time.

```
#[tauri::command]
#[specta::specta]
pub async fn run_query(
    state: tauri::State<'_, AppState>,
    session_id: SessionId,
    sql: String,
    approval: Option<ApprovalToken>,
    run_id: RunId,
    on_rows: tauri::ipc::Channel<RowBatch>,
) -> Result<Vec<StatementOutcome>, AppError> {
    state.sessions.run_user(session_id, &sql, approval, run_id, on_rows, &state.approvals).await
        .map_err(AppError::from)
}
```

```
import { Channel } from '@tauri-apps/api/core';
import { commands } from '../bindings';

const rows = new Channel<RowBatch>();
rows.onmessage = (batch) => grid.append(batch.rows);       // virtual grid grows as rows arrive
const res = await commands.runQuery(sessionId, sql, null, runId, rows);
if (res.status === 'error' && res.error.code === 'ApprovalRequired')
    openConfirmDialog(res.error);
```

7.3 Errors

`AppError` is a serializable enum with a stable `code` (for example `NotConnected`, `ApprovalRequired`, `ApprovalMismatch`, `ReadOnlyProfile`, `Timeout`, `Cancelled`, `Database`, `Tls`, `Ssh`, `Ai`) and a human message. Database error details (SQLSTATE, position) are passed through so the editor can underline the error.

7.4 Command catalogue

Command	Purpose
`list_profiles`, `save_profile`, `delete_profile`	Profile CRUD (secrets written to keychain, never returned)
`set_secret`, `clear_secret`	Store or remove a password, passphrase or API key
`test_connection`	Connect (with tunnel and TLS), report server version and TLS status
`connect`, `disconnect`	Open or close a session
`list_schemas`, `list_objects`, `columns`, `table_structure`	Metadata for the explorer, editor completion and AI
`table_page`	Server-side paged, sorted, filtered table data (bound parameters)
`table_count`	Time-boxed row count
`prepare_query`	Classify a script; return per-statement verdicts and risk reasons
`approve_query`	Called by the confirm dialog; returns a single-use token for that exact SQL
`run_query`	Execute with streaming results (Channel)
`cancel_run`	Cancel one run
`begin_tx`, `commit_tx`, `rollback_tx`, `tx_state`	Manual-commit mode
`explain_query`	JSON plan for the visual plan viewer
`export_query`, `export_table`	Stream to CSV, JSON, XLSX or SQL file chosen with the dialog plugin
`import_csv`	Column mapping, preview, batched inserts after confirmation
`history_list`, `history_clear`, `drafts_*`	History and saved queries
`ai_send`, `ai_cancel`, `ai_review`, `ai_cancel_query`, `ai_forget`	AI turns, review decisions, memory
`audit_list`, `audit_export`	Audit log
`settings_get`, `settings_set`	Preferences

Table 7.2: Commands registered by `studio-app`.

CHAPTER 8

Key Flows

8.1 Connecting through an SSH tunnel with TLS

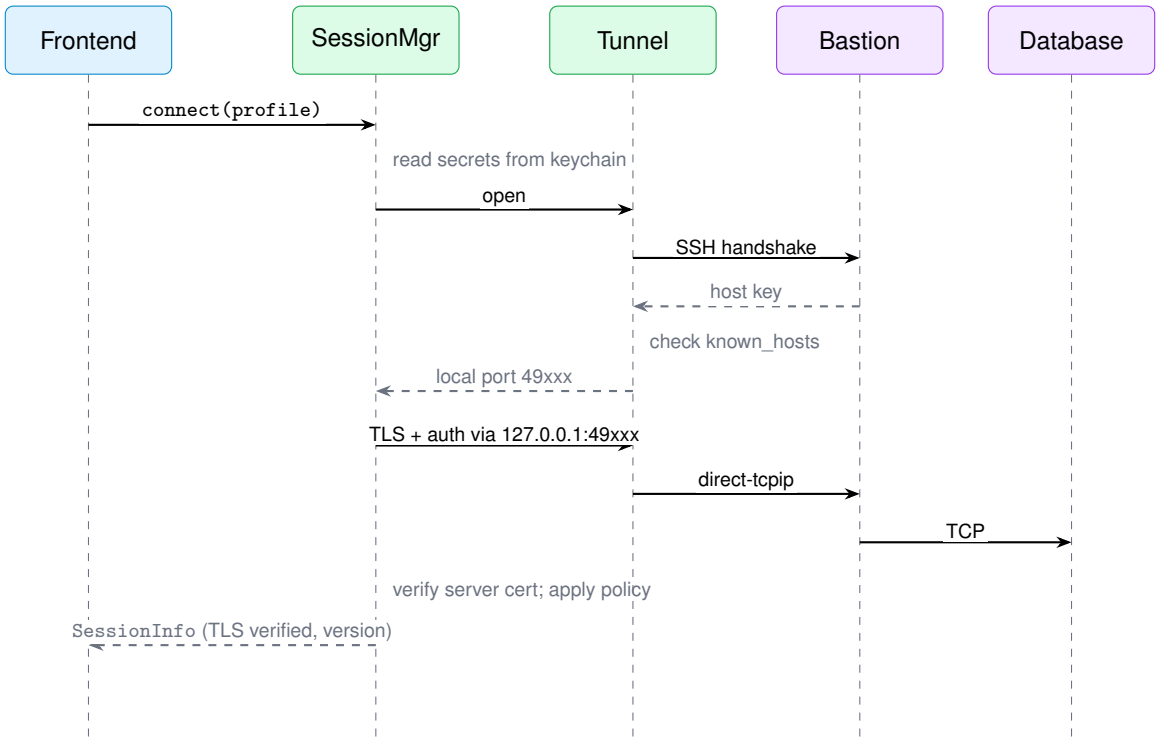


Figure 8.1: TLS runs end-to-end through the tunnel, so the database certificate is still verified.

8.2 Running a query with streaming results

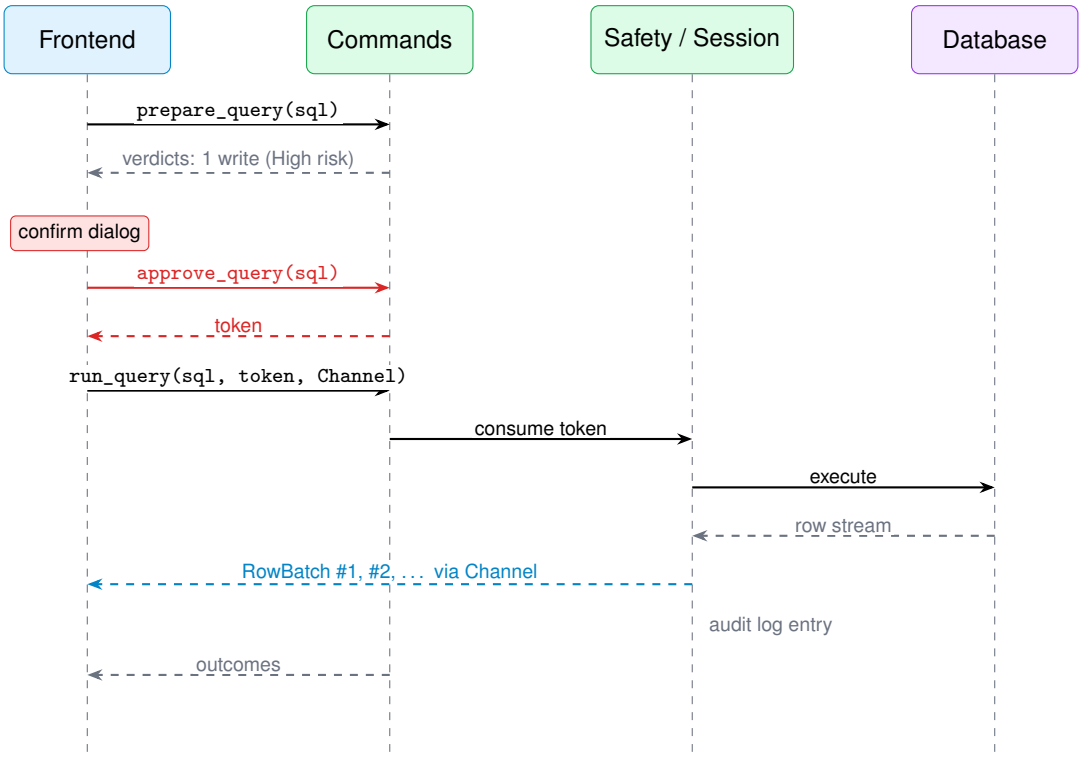


Figure 8.2: For a read, `approve_query` is skipped. The grid renders the first batch immediately.

8.3 An AI turn with one approved write

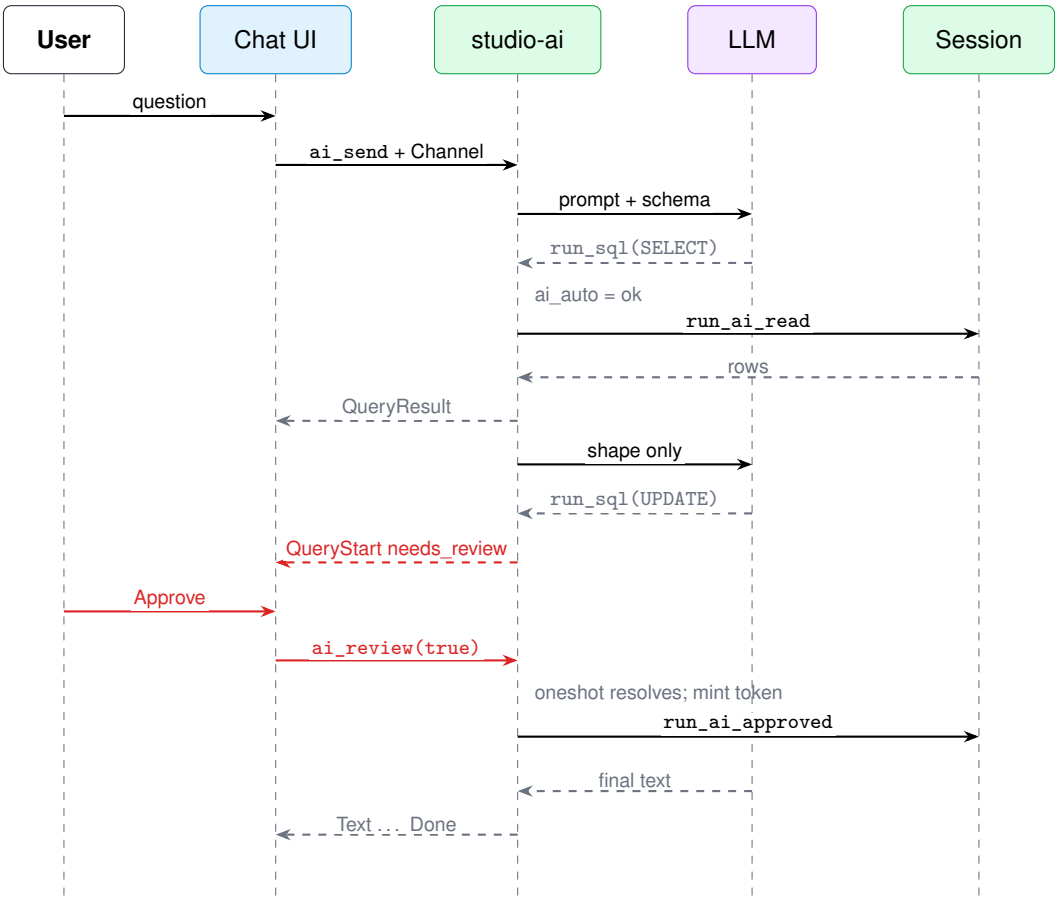


Figure 8.3: The model can request a write; only the user's click can complete the review.

CHAPTER 9

Security Architecture

9.1 Threat model

Threat	Mitigation
Model writes data on its own	No write path without a oneshot completed by a UI click; approvals bound to SQL hash and session
Prompt injection stored in table data	Rows not sent by default; when sent, labelled untrusted; tools cannot widen their own permissions
Script hides a write (`SELECT 1; DROP \ldots`)	Parsed per statement; AI limited to one statement
Side-effecting reads	Function deny-list; read-only transaction; timeouts
Compromised or buggy webview	Capabilities whitelist commands; CSP; no shell/fs plugins granted; secrets never sent to the UI; Rust re-validates every input
Network attacker	rustls with certificate verification by default; SSH host-key pinning
Stolen laptop / other user	Secrets in OS keychain; optional SQLCipher; files owner-only
Malicious update	Updater verifies signatures with the public key embedded in the app
Vulnerable dependencies	`cargo-deny` and `cargo-audit` in CI; pinned lockfile

Table 9.1: Threats and mitigations.

9.2 Tauri capabilities

Tauri 2 denies every command and plugin API unless a capability file grants it. The main window gets only what the app uses:

```
// src-tauri/capabilities/main.json
{
  "identifier": "main",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "dialog:allow-open",
    "dialog:allow-save",
    "updater:default",
    "process:allow-restart"
  ]
}
```

App commands registered through the builder are also covered by the app's permission manifest. Plugins such as `shell` and `fs` are not installed at all; file export goes through a Rust command that writes to the path the user chose in the save dialog.

9.3 Content Security Policy

Set a strict CSP in `tauri.conf.json` (`app.security.csp`): scripts only from the app, `connect-src` limited to Tauri's IPC, no remote frames, no inline scripts. Links from AI replies are opened with the opener plugin only for `https:` and `mailto:` schemes.

9.4 TLS

Mode	Behaviour
`disable`	Plain TCP. Allowed only for `localhost` unless the user confirms.
`require`	Encrypt; do not verify (legacy servers). Shown with a warning badge.
`verify-ca`	Verify the chain against system roots or a chosen CA file.
`verify-full`	Verify chain *and* host name. **Default for new profiles.**

Table 9.2: TLS modes, using PostgreSQL's naming for all engines.

System roots come from `rustls-native-certs`; cloud CA bundles (RDS, Azure, Supabase) can be attached per profile. Client certificates are supported for mutual TLS.

CHAPTER 10

Frontend

10.1 Reusing the React app

The existing React code moves to `src/` almost unchanged. The work is in three places:

1. **API adapter.** Replace `window.electronAPI.*` with `commands.*` from the generated bindings. Put a thin `src/api/` layer in between so components do not depend on transport details.
2. **Streaming.** Query results arrive as batches on a `Channel`. The results store appends batches; TanStack Virtual renders only visible rows, so 1 million rows stay smooth. The grid shows "fetching… 120,000 rows" while the stream is open and offers Stop.
3. **AI events.** `ChatContext` creates a `Channel<AiEvent>` per turn instead of listening to a global event and filtering by request id.

10.2 State

Keep React context for app-wide state, but consider **TanStack Query** for server state (profiles, schemas, structures): caching, invalidation after DDL, and request de-duplication come for free. `tauri-specta` ecosystem tools can generate query helpers for commands.

10.3 New UI for production features

- Environment colour band and "READ-ONLY" badge in the title bar.
- Transaction bar: Auto/Manual toggle, pending-changes indicator, Commit, Rollback.
- Visual `EXPLAIN` plan tree with cost hot spots.
- SSH and TLS tabs in the connection dialog, with host-key fingerprint confirmation.
- Import wizard (CSV mapping, preview, dry run) and export dialog.
- Audit log viewer; data-masking rules editor.
- Update notification with release notes.

CHAPTER 11

Production Features Map

Feature	Crate(s)	Implementation notes
TLS verification	drivers	rustls verifier; modes as in Chapter 8; CA file per profile
SSH tunnelling	studio-tunnel	russh direct-tcpip; key, agent, password auth; known_hosts
Read-only / prod profiles	studio-session	policy check before classification result is used; read-only default transaction
Manual transactions	studio-session, drivers	pinned connection; commit/rollback commands; warn on close with uncommitted work
Streaming results	studio-db, drivers	RowStream to mpsc to Channel; backpressure via bounded channel
Full export	studio-app	stream query into CSV/JSON/XLSX (`rust_xlsxwriter`) or SQL INSERTs, off the UI thread
CSV import	studio-app, studio-session	`csv` crate; batched multi-row INSERT inside one transaction; approval once for the import
Backup / restore	studio-app	invoke `pg_dump` / `mysqldump` if installed, with progress events
Visual EXPLAIN	drivers	PostgreSQL `FORMAT JSON`; MySQL `FORMAT=JSON`
SQLite and more engines	driver-*	new crate implementing `Driver`; register in app
AI explain / optimise / fix	studio-ai	editor actions that start a turn with the selection and last error
Index suggestions	studio-ai	plan + structure to model; proposal goes through review
Data masking	studio-store, studio-ai	per-column rules; applied to AI sharing and optionally to the grid
Audit log	studio-store	hash-chained rows; export
Encrypted local data	studio-store	SQLCipher; key in keychain
Auto-update	studio-app	`tauri-plugin-updater`; signed artifacts
Crash reports (opt-in)	studio-app	panic hook + `sentry` crate; never include SQL text by default
Secrets managers	studio-store	optional providers: 1Password CLI, Vault, AWS Secrets Manager

Table 11.1: Where each production feature lives.

CHAPTER 12

Build, Release and Operations

12.1 Development

```
rustup toolchain install stable
cargo install tauri-cli --version "^2"      # provides ‘cargo tauri‘
npm ci
cargo tauri dev                             # Vite dev server + Rust app with reload
cargo test --workspace                      # all crate tests
cargo clippy --workspace -- -D warnings
```

12.2 Bundling

Platform	Bundles	Signing
macOS	`.app`, `.dmg` (universal or per-arch)	Developer ID certificate + notarisation
Windows	NSIS `.exe` and/or `.msi`	Authenticode certificate (OV/EV or cloud signing)
Linux	AppImage, `.deb`, `.rpm`	Updater signature; optional repository signing

Table 12.1: Release targets via `cargo tauri build`.

Linux uses WebKitGTK as its webview; test the grid and Monaco there early, since rendering and performance differ from WebView2 (Windows) and WKWebView (macOS).

12.3 Auto-update

Generate an updater key pair once (`cargo tauri signer generate`); keep the private key in CI secrets and the public key in `tauri.conf.json` under `plugins.updater.pubkey`. Releases publish a JSON manifest (GitHub Releases works). The app checks on start and daily, shows release notes, downloads, verifies the signature and restarts.

12.4 Continuous integration

- **ci.yml** on every pull request: `cargo fmt -check`, `cargo clippy`, `cargo test -workspace` (with PostgreSQL and MySQL service containers), `cargo deny check`, frontend `tsc` and unit tests, and a check that `bindings.ts` is up to date.
- **release.yml** on tags: matrix over macOS (arm64, x64), Windows and Ubuntu using `tauri-apps/tauri-action`; signs, notarises, uploads artifacts and the updater manifest.

12.5 Observability

Use `tracing` with `tracing-subscriber` writing to a rotating log file in the app log directory (`tracing-appender`). Log events, durations and error codes; never log SQL parameters, passwords or result rows. A “Copy diagnostics” button collects version, OS, recent log lines and settings (with secrets removed) for bug reports.

CHAPTER 13

Testing Strategy

Level	What	Tools
Unit	classifier rules, approval book, value encoding, filter builder, agent loop with a fake provider	`cargo test`, `insta` snapshots, `proptest`
Fuzz	classifier never panics; no write marked ai_auto	`cargo-fuzz`
Integration	each driver against real servers: metadata, streaming, cancel, read-only enforcement, TLS	`testcontainers` (PostgreSQL 14–17, MySQL 8.x, MariaDB)
Tunnel	forward through a containerised OpenSSH server; host-key change rejected	`testcontainers`
Frontend	components, grid logic, chat event reducer	Vitest, Testing Library
End-to-end	connect, run, edit with confirm, AI review flow	WebDriver via `tauri-driver`
Performance	1M-row stream memory and time; UI frame rate	benchmarks in CI on a fixed runner

Table 13.1: Test pyramid.

Safety tests are release blockers

A “safety suite” runs AI-path statements against real databases inside the read-only path and asserts that the database refused every write the classifier might have missed. Any failure blocks a release.

CHAPTER 14

Migration Plan and Roadmap

14.1 From the Electron version

1. **Freeze the contract.** Write down the current IPC surface (`ElectronAPI` in `shared/types.ts`) and port those types to `studio-types` with `specta`.
2. **Port tests first.** Move the classifier, agent and filter test cases into Rust test tables so behaviour is preserved.
3. **Bring up the shell.** Tauri app loading the existing React build, with a temporary adapter that maps old API names to new commands.
4. **Replace services one by one:** storage, then sessions and drivers, then safety, then AI.
5. **Migrate user data.** On first launch, import `studio-storage.json` and `ai-memory.json` into `studio.db`; move secrets into the keychain; keep a backup.

14.2 Milestones

Phase	Duration	Deliverables
0. Foundations	2 weeks	Workspace, CI, Tauri shell with React, specta bindings, store with migrations, keyring
1. Client parity	6 weeks	PostgreSQL and MySQL drivers, streaming grid, explorer, structure, table editing, history, drafts, classifier + approvals
2. AI parity	3 weeks	Provider adapter, agent loop, review gate, memory, fallbacks
3. Production hardening	4 weeks	TLS modes, SSH tunnels, read-only/prod profiles, manual transactions, audit log, signed builds, updater, Linux
4. Differentiators	4 weeks	SQLite driver, export/import, visual EXPLAIN, AI explain/optimise/fix, index suggestions, masking
5. Team features	ongoing	Encrypted store, secret-manager providers, shared query library, more engines

Table 14.1: Indicative roadmap for one experienced developer; adjust to your pace.

14.3 Risks

- **Webview differences** (especially WebKitGTK on Linux) can affect Monaco and the grid. Mitigate by testing all three platforms from Phase 0.
- **Parser coverage**: `sqlparser` accepts many but not all dialect features. Unparseable SQL fails closed (needs confirmation), which is safe but may annoy users; track and report gaps.
- **Fast-moving AI crates**: isolate `rig-core` behind the `Provider` trait.
- **Rust learning curve**: start with `studio-store` and `studio-safety`, which have little async complexity, before drivers and the agent.

CHAPTER A

Dependency Reference

Versions noted are the latest seen when this document was written; confirm with `cargo add` when starting, and pin through `[workspace.dependencies]`.

Crate	Version	Role
`tauri, tauri-build`	2.11.x	Desktop shell, IPC, bundler
`tauri-plugin-updater, -dialog, -process, -opener`	2.x	Updates, file dialogs, restart, safe link opening
`tauri-specta, specta, specta-typescript`	2.0 RC	Typed TypeScript bindings for commands and events
`tokio, tokio-util`	1.x	Async runtime, CancellationToken
`tokio-postgres, deadpool-postgres`	0.7.x	PostgreSQL driver and pool
`tokio-postgres-rustls, rustls, rustls-native-certs`	latest	TLS with verification
`mysql_async`	latest	MySQL / MariaDB driver
`rusqlite (bundled)`	latest	SQLite driver for user databases
`sqlx (sqlite, migrate)`	0.8.6	App's own database
`sqlparser (visitor)`	0.61+	SQL parsing for safety
`russh`	0.63.x	SSH tunnels
`keyring`	3.6.x	OS credential stores
`secrecy, zeroize`	latest	Secret handling in memory
`rig-core`	0.46.x	LLM providers, streaming, tool schemas
`serde, serde_json`	1.x	Serialization
`thiserror, anyhow`	latest	Errors (libraries / app)
`dashmap`	latest	Concurrent maps
`sha2, rand`	latest	Approval hashing and tokens
`csv, rust_xlsxwriter`	latest	Import / export
`tracing, tracing-subscriber, tracing-appender`	latest	Logging
`testcontainers, insta, proptest`	latest	Testing

Table A.1: Main dependencies.

CHAPTER B

Example Workspace Manifest

```
[workspace]
resolver = "2"
members = ["crates/*", "src-tauri"]

[workspace.package]
edition = "2021"
license = "Apache-2.0"

[workspace.dependencies]
tokio        = { version = "1", features = ["rt-multi-thread", "macros", "sync", "time", "net",
    "io-util"] }
tokio-util   = "0.7"
serde        = { version = "1", features = ["derive"] }
serde_json   = "1"
thiserror    = "2"
async-trait  = "0.1"
dashmap      = "6"
tracing      = "0.1"
specta       = { version = "=2.0.0-rc.NN", features = ["derive"] }   # use the exact pin from
    tauri-specta's README
sqlparser    = { version = "0.61", features = ["visitor"] }
keyring      = { version = "3", features = ["apple-native", "windows-native",
    "sync-secret-service"] }
russh        = { version = "0.63", features = ["aws-lc-rs"] }
rig-core     = "0.46"
# internal crates
studio-types   = { path = "crates/studio-types" }
studio-db      = { path = "crates/studio-db" }
studio-safety  = { path = "crates/studio-safety" }
studio-session = { path = "crates/studio-session" }
studio-tunnel  = { path = "crates/studio-tunnel" }
studio-store   = { path = "crates/studio-store" }
studio-ai      = { path = "crates/studio-ai" }

[profile.release]
lto = "thin"
codegen-units = 1
strip = true
```

CHAPTER C

Glossary

Capability
: A Tauri 2 file that grants a window permission to use specific commands and plugin APIs.

Channel
: A Tauri IPC object passed to a command that lets Rust send an ordered stream of messages to the frontend for that call.

Fail closed
: When a check cannot decide, it chooses the safe outcome (here: require approval).

Oneshot
: A Tokio channel that carries exactly one value; used to wait for a review decision.

RAII
: Resource cleanup tied to a value's lifetime; dropping a `Tunnel` closes it.

Read-only transaction
: A transaction the database itself refuses to write in.

Webview
: The operating system's browser engine that renders the UI: WKWebView (macOS), WebView2 (Windows), WebKitGTK (Linux).